P1967R4

`#embed` - a scannable, tooling-friendly binary resource inclusion mechanism

Published Proposal, 2021-06-15

Authors:

JeanHeyd Meneide

Shepherd (Shepherd's Oasis LLC)

Paper Source:

GitHub

Implementation:

GitHub

Issue Tracking:

GitHub

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

EWG

Abstract

Pulling binary data into a program often involves external tools and build system coordination. Many programs need binary data such as images, encoded text, icons and other data in a specific format. Current state of the art for working with such static data in C includes creating files which contain solely string literals, directly invoking the linker to create data blobs to access through carefully named extern variables, or generating large brace-delimited lists of integers to place into arrays. As binary data has grown larger, these approaches have begun to have drawbacks and issues scaling. From parsing 5 megabytes worth of integer literal expressions into AST nodes to arbitrary string literal length limits in compilers, portably putting binary data in a C program has become an arduous task that taxes build infrastructure and compilation memory and time. This proposal provides a flexible preprocessor directive for making this data available to the user in a straightforward manner.

Table of Contents

§ 1. Changelog

§ 1.1. Revision 4 - June 15th, 2021

— Vastly improve C++ wording after June 3rd, 2021 discussion.

— Change syntax after comments from implementers of scanners / dependency trackers, and comments from implementers that were supported by users.

— Add support for "named parameter" implementation extensions.

§ 1.2. Revision 3 - April 15th, 2021 (WG21), May 15th, 2021 (WG14)

— Added post C meeting fixes to prepare for hopeful success next meeting.

— Added 2 more examples to C and C++ wording.

— Vastly improved wording and reduced ambiguities in syntax and semantics.

— Fixed various wording issues.

§ 1.3. Revision 2 - October 25th, 2020

— Added post C++ meeting notes and discussion.

— Removed type or bit specifications from the `#embed` directive.

— Moved "Type Flexibility" section and related notes to the Appendix as they are now unpursued.

§ 1.4. Revision 1 - April 10th, 2020

— Added post C meeting notes and discussion.

— Added discussion of potential endianness.

— Improved wording section at the end to be more detailed in handling preprocessor (which does not understand types).

§ 1.5. Revision 0 - January 5th, 2020

— Initial release.

§ 2. Polls & Votes

The votes for the C++ Committee are as follows:

— SF: Strongly in Favor

— F: In Favor

— N: Neutral

— A: Against

— SA: Strongly Against

The votes for the C Committee are as follows:

— Y: Ye

— N: Nay

— A: Abstain

§ 2.1. December 2020 Virtual C Meeting

"Do we want to allow #embed to appear in any context that is different from an initialization of a character array?"

Y	N	A
5	8	6

"Leaning in the direction of no but not clear." The paper author after consideration chose to keep this as-is right now. Discussion of the feature meant that trying to ban this from different contexts meant that a naïve, separated-preprocessor implementation would be banned and it would require special compiler magic to diagnose. Others pointed out that just trying to leave it "unspecified whether it works outside of the initialization of an array or not" is very dangerous to portability. The author agrees with this assessment and therefore will leave it as-is. The goal of this feature is to enable implementers to use the magic if they so choose, as an implementation detail and a Quality of Implementation selling point. Vendors who provide a simple expansion may not see improvements to throughput and speed of translation but that is their choice as an implementer. Therefore, we cannot do anything which would require them or any preprocessor implementation to traffic in magic directives unless they want to.

§ 2.2. September 2020 Virtual C++ EWG Meeting

"We want `#embed [optional limit] header-name` (no type name, no other specification) as a feature."

SF	F	N	A	SA
2	16	3	0	1

This vote gained the most consensus in the Committee. While there were some individuals who wanted to be able to specify a type, there was stronger interest in not specifying a type at all and always producing a list of integer literals suitable to be used anywhere an `comma-separated list` was valid.

"We want to explore allowing an optional sequence of tokens to specify a type to `#embed`."

SF	F	N	A	SA
1	9	4	4	3

Further need was also expressed for `constexpr` of different types of variables, so we would rather focus that ability into a sister feature, [`std::embed`](/_vendor/future_cxx). There was also an expression to augment `std::bitcast<...>(...)` to handle arrays of data, which would be a follow-on proposal. There was a great amount of interest in the `std::bitcast` direction, which means a paper should be written to follow up on it.

§ 2.3. April 2020 Virtual C Meeting

"We want to have a proper preprocessor `#embed ...` over a `#pragma _STDC embed ...`-based directive."

This had UNANIMOUS CONSENT to pursue a proper preprocessor directive and NOT use the `#pragma` syntax. It is noted that the author deems this to be the best decision!

The following poll was later superceded in the C and C++ Committees.

"We want to specify embed as using `#embed [bits-per-element] header-name` rather than `#embed [pp-tokens-for-type] header-name`." (2-way poll.)

Y	N	A
10	2	3

— Y: 10 bits-per-element (Ye)

— N: 2 type-based (Nay)

— A: 4 Abstain (Abstain)

This poll will be a bit harder to accommodate properly. Using a *`constant-expression`* that produces a numeric constant means that the max-length specifier is now ambiguous. The syntax of the directive may need to change to accommodate further exploration.

§ 3. Introduction

For well over 40 years, people have been trying to plant data into executables for varying reasons. Whether it is to provide a base image with which to flash hardware in a hard reset, icons that get

packaged with an application, or scripts that are intrinsically tied to the program at compilation time, there has always been a strong need to couple and ship binary data with an application.

C does not make this easy for users to do, resulting in many individuals reaching for utilities such as `xxd`, writing python scripts, or engaging in highly platform-specific linker calls to set up `extern` variables pointing at their data. Each of these approaches come with benefits and drawbacks. For example, while working with the linker directly allows injection of vary large amounts of data (5 MB and upwards), it does not allow accessing that data at any other point except runtime. Conversely, doing all of these things portably across systems and additionally maintaining the dependencies of all these resources and files in build systems both like and unlike `make` is a tedious task.

Thusly, we propose a new preprocessor directive whose sole purpose is to be `#include`, but for binary data: `#embed`.

§ 3.1. Motivation

The reason this needs a new language feature is simple: current source-level encodings of "producing binary" to the compiler are incredibly inefficient both ergonomically and mechanically. Creating a brace-delimited list of numerics in C comes with baggage in the form of how numbers and lists are formatted. C's preprocessor and the forcing of tokenization also forces an unavoidable cost to lexer and parser handling of values.

Therefore, using arrays with specific initialized values of any significant size becomes borderline impossible. One would think this old problem would be work-around-able in a succinct manner. Given how old this desire is (that comp.std.c thread is not even the oldest recorded feature request), proper solutions would have arisen. Unfortunately, that could not be farther from the truth. Even the compilers themselves suffer build time and memory usage degradation, as contributors to the LLVM compiler ran the gamut of the biggest problems that motivate this proposal in a matter of a week or two earlier this very year. Luke is not alone in his frustrations: developers all over suffer from the inability to include binary in their program quickly and perform exceptional gymnastics to get around the compiler's inability to handle these cases.

C developer progress is impeded regarding the inability to handle this use case, and it leaves both old and new programmers wanting.

§ 3.2. But *How* Expensive Is This?

Many different options as opposed to this proposal were seriously evaluated. Implementations were attempted in at least 2 production-use compilers, and more in private. To give an idea of usage and

size, here are results for various compilers on a machine with the following specification:

— Intel Core i7 @ 2.60 GHz

— 24.0 GB RAM

— Debian Sid or Windows 10

— Method: Execute command hundreds of times, stare extremely hard at `htop`/Task Manager

While `time` and `Measure-Command` work well for getting accurate timing information and can be run several times in a loop to produce a good average value, tracking memory consumption without intrusive efforts was much harder and thusly relied on OS reporting with fixed-interval probes. Memory usage is therefore approximate and may not represent the actual maximum of consumed memory. All of these are using the latest compiler built from source if available, or the latest technology preview if available. Optimizations at `-O2` (GCC & Clang style)/`/O2 /Ob2` (MSVC style) or equivalent were employed to generate the final executable.

§ 3.2.1. Speed

Strategy	40 kilobytes	400 kilobytes	4 megabytes	40 megabytes
`#embed` GCC	0.236 s	0.231 s	0.300 s	1.069 s
`xxd`-generated GCC	0.406 s	2.135 s	23.567 s	225.290 s
`xxd`-generated Clang	0.366 s	1.063 s	8.309 s	83.250 s
`xxd`-generated MSVC	0.552 s	3.806 s	52.397 s	Out of Memory

§ 3.2.2. Memory Size

Strategy	40 kilobytes	400 kilobytes	4 megabytes	40 megabytes
`#embed` GCC	17.26 MB	17.96 MB	53.42 MB	341.72 MB
`xxd`-generated GCC	24.85 MB	134.34 MB	1,347.00 MB	12,622.00 MB
`xxd`-generated Clang	41.83 MB	103.76 MB	718.00 MB	7,116.00 MB
`xxd`-generated MSVC	~48.60 MB	~477.30 MB	~5,280.00 MB	Out of Memory

§ 3.2.3. Analysis

The numbers here are not reassuring that compiler developers can reduce the memory and compilation time burdens with regard to large initializer lists. Furthermore, privately owned compilers and other static analysis tools perform almost exponentially worse here, taking vastly more memory and

thrashing CPUs to 100% for several minutes (to sometimes several hours if e.g. the Swap is engaged due to lack of main memory). Every compiler must always consume a certain amount of memory in a relationship directly linear to the number of tokens produced. After that, it is largely implementation-dependent what happens to the data.

The GNU Compiler Collection (GCC) uses a tree representation and has many places where it spawns extra "garbage", as its called in the various bug reports and work items from implementers. There has been a 16+ year effort on the part of GCC to reduce its memory usage and speed up initializers (C Bug Report and C++ Bug Report). Significant improvements have been made and there is plenty of room for GCC to improve here with respect to compiler and memory size. Somewhat unfortunately, one of the current changes in flight for GCC is the removal of all location information beyond the 256th initializer of large arrays in order to save on space. This technique is not viable for static analysis compilers that promise to recreate source code exactly as was written, and therefore discarding location or token information for large initializers is not a viable cross-implementation strategy.

LLVM's Clang, on the other hand, is much more optimized. They maintain a much better scaling and ratio but still suffer the pain of their token overhead and Abstract Syntax Tree representation, though to a much lesser degree than GCC. A bug report was filed but talk from two prominent LLVM/Clang developers made it clear that optimizing things any further would require an extremely large refactor of parser internals with a lot of added functionality, with potentially dubious gains. As part of this proposal, the implementation provided does attempt to do some of these optimizations, and follows some of the work done in this post to try and prove memory and file size savings. (The savings in trying to optimize parsing large array literals were "around 10%", compared to the order-of-magnitude gains from `#embed` and similar techniques).

Microsoft Visual C (MSVC) scales the worst of all the compilers, even when given the benefit of being on its native operating system. Both Clang and GCC outperform MSVC on Windows 10 or WINE as of the time of writing.

Linker tricks on all platforms perform better with time (though slower than `#embed` implementation), but force the data to be optimizer-opaque (even on the most aggressive "Link Time Optimization" or "Whole Program Optimization" modes compilers had). Linker tricks are also exceptionally non-portable: whether it is the `incbin` assembly command supported by certain compilers, specific invocations of `rc.exe`/`objcopy` or others, non-portability plagues their usefulness in writing Cross-Platform C (see Appendix for listing of techniques). This makes C decidedly unlike the "portable assembler" advertised by its proponents (and my Professors and co-workers).

§ 4. Design

There are two design goals at play here, sculpted to specifically cover industry standard practices with build systems and C programs.

The first is to enable developers to get binary content quickly and easily into their applications. This can be icons/images, scripts, tiny sound effects, hardcoded firmware binaries, and more. In order to support this use case, this feature was designed for simplicity and builds upon widespread existing practice.

The second is extensibility. We recognize that talking to arbitrary places on either the file system, network, or similar has different requirements. After feedback from an implementer about syntax for extensions, we reached out to various users of the beta builds or custom builds using `#embed`-like things. It turns out many of them have needs that, since they are the ones building and in some cases patching over/maintaining their compiler, have needs for extensible attributes that can be passed to `#embed` directives. Therefore, we structured the syntax in a way that is favorable to "simple" scanning tools but powerful enough to handle arbitrary directives and future extension points.

§ 4.1. Goal: Simplicity and Familiarity

Providing a directive that mirrors `#include` makes it natural and easy to understand and use this new directive. It accepts both chevron-delimited (`<>`) and quote-delimited (`""`) strings like `#include` does. This matches the way people have been generating files to `#include` in their programs, libraries and applications: matching the semantics here preserves the same mental model. This makes it easy to teach and use, since it follows the same principles:

```
/* default is unsigned char */
const unsigned char icon_display_data[] = {
                #embed "art.png"
};

/* specify any type which can be initialized form integer constant expressi
const char reset_blob[] = {
                #embed "data.bin"
};
```

Because of its design, it also lends itself to being usable in a wide variety of contexts and with a wide variety of vendor extensions. For example:

```
/* attributes work just as well */
const signed char aligned_data_str[] __attribute__ ((aligned (8))) = {
                #embed "attributes.xml"
};
```

The above code obeys the alignment requirements for an implementation that understands GCC directives, without needing to add special support in the `#embed` directive for it: it is just another array initializer, like everything else.

4.1.1. Existing Practice - Search Paths

It follows the same implementation experience guidelines as `#include` by leaving the search paths implementation defined, with the understanding that implementations are not monsters and will generally provide `-fembed-path`/`-fembed-path=` and other related flags as their users require for their systems. This gives implementers the space they need to serve the needs of their constituency.

4.1.2. Existing Practice - Discoverable and Distributable

Build systems today understand the make dependency format, typically through use of the compiler flags `-(M)MD` and friends. This sees widespread support, from CMake, Meson and Bazel to ninja and make. Even VC++ has a version of this flag -- `/showIncludes` -- that gets parsed by build systems.

This preprocessor directive fits perfectly into existing build architecture by being discoverable in the same way with the same tooling formats. It also blends perfectly with existing distributed build systems which preprocess their files with `-frewrite-includes` before sending it up to the build farm, as `distcc` and `icecc` do.

4.2. Syntax

The syntax for this feature is for an extensible preprocessor directive. The general form is:

```
# embed <header-name>|"header-name" attribute...
```

where `attribute` refers to the syntax of `no_arg`/`with_arg(values, ...)`/`vendor::no_arg`/`vendor::with_arg(values, ...)` that is already part of the grammar.

The attributes here do not have the typical double-bracket delimiters here, and simply function as a way to provide named parameters to the `#embed` directive.

This syntax keeps the header-name, enclosed in angle brackets or quotation marks, first to allow a "simple" preprocessing tool to quickly scan for all the necessary dependency names without having to parse any of the names or parameters that come after. Both standard names and vendor/implementation-specific names can also be accommodated in the list of naked attributes, allowing for specific vendor extensions in a consistent manner while the standard can take the normal `foo` names.

§ 4.2.1. Parameters

One of the things that's critical about `#embed` is that, because it works with binary resources, those resources have characteristics very much different from source and header files present in a typical filesystem. There may be need for authentication (possibly networked), permission, access, additional processing (new-line normalization), and more that can be somewhat similarly specified through the implementation-defined parameters already available through the C and C++ Standards' "`fopen`" function.

However, adding a "mode" string similar to `fopen`, while extensible, is archaic and hard to check. Therefore, the syntax allows for multiple "named expressions", encapsulated in parentheses, and marked with `::` as a form of "namespacing" identifiers similar to `[[vendor::attr]]` attribute-style syntax. However, parameters do not have the balanced square bracket `[[]]` delimiters, and just use the `vendor::attr` form with an optional parentheses-enclosed list of arguments.

Some example attributes including interpreting the binary data as "text" rather than a bitstream with `clang::text`, providing authenticated access with `fs::auth("username", "password")`, `yosys::type(hardware_entry)` to change the element of each entry produced, and more. These are all things vendors have indicated they might support for their use cases.

§ *4.2.1.1. Limit Parameter*

The earliest adopters and testers of the implementation reported problems when trying to access POSIX-style `char` devices and pseudo-files that do not have a logical limitation. These "infinity files" served as the motivation for introducing the "limit" parameter; there are a number of resources which are logically infinite and thusly having a compiler read all of the data would result an Out of Memory error, much like with `#include` if someone did `#include "/dev/urandom"`.

The limit parameter is specified after the resource name in `#embed`, like so:

```
const int please_dont_oom_kill_me[] = {
        #embed "/dev/urandom" limit(512)
};
```

This prevents locking compilers in an infinite loop of reading from potentially limitless resources. Note the parameter is a hard upper bound, and not an exact requirement. A resource may expand to a 16-element list rather than a 512-element list, and that is entirely expected behavior. The limit is the number of elements allowed up to the maximum for this type.

§ *4.2.1.2. Non-Empty Prefix and Suffix*

Something pointed out by others using this preprocessor directive is a problem similar to `__VA_ARGS__`: when placing this parameter with other tokens before or after the `#embed` directive, it sometimes made it hard to properly anticipate whether a file was empty or not.

The `#embed` proposal includes a prefix and suffix entry that applies if and only if the resource is non-empty:

```
const unsigned char null_terminated_file_data[] = {
        #embed "might_be_empty.txt" \
                prefix(0xEF, 0xBB, 0xBF, ) /* UTF-8 BOM */ \
                suffix(,)
        0 // always null-terminated
};
```

`prefix` and `suffix` only work if the `#embed` resource is not empty. If a user wants a prefix or suffix that appears unconditionally, they can simply just type the tokens they want before and after: there is nothing to be gained from adding a standards-mandated prefix and suffix that works in both the empty and non-empty case.

§ 4.3. Constant Expressions

Both C and C++ compilers have rich constant folding capabilities. While C compilers only acknowledge a fraction of what is possible by larger implementations like MSVC, Clang, and GCC, C++ has an entire built-in compile-time programming bit, called `constexpr`. Most typical solutions

cannot be used as constant expressions because they are hidden behind run-time or link-time mechanisms (`objcopy`, or the resource compiler `rc.exe` on Windows, or the static library archiving tools). This means that many algorithms and data components which could strongly benefit from having direct access to the values of the integer constants do not because the compiler cannot "see" the data, or because Whole Program Optimization cannot be aggressive enough to do anything with those values at that point in the compilation (i.e., during the final linking stage).

This makes `#embed` especially powerful, since it guarantees these values are available as-if it was written by as a sequence of integers whose values fit within an `unsigned char`.

§ 4.4. `unsigned char` values, exactly

The specification requires that each value is first cast to `unsigned char`, by means of a `static_cast<unsigned char>(generated_value)` in C++, or a `(unsigned char)generated_value` in C. The reason for this is to prevent each value from being interpreted as a signed `int` value instead, which can result in out-of-bound errors when e.g. a `signed char` or an implementation-defined-to-be-signed `char` is initialized with the values in the comma-delimited list.s

§ 4.5. `__has_embed`

C and C++ are both working on (or already support) as `__has_include` directive. It makes sense to have an analogous `__has_embed` identifier. It can take a `__has_embed( "header-name" ... )` or `__has_embed (<header-name> ... )` resource name identifier, as well as additional arguments to let vendors pass in any additional arguments they need to properly access the file (following the same attribute-like parameters passed to the directive). `__has_embed` evaluates to:

— `0` if the resource cannot be found;

— `1` if the resource is found and it is not empty;

This may raise questions of "TOCTTOU" (Time of Check to Time of Use) problems, but we already have these problems between `__has_include` and `#include`. For example, the Clang compiler uses a `FileManager` and `SourceManager` abstractions which cache files. GCC's "libcpp" will cache already-opened files (up to a limit). Any TOCTTOU problems have already been managed and provided for using the current `#include` infrastructure of these compilers, and if any compiler wants a more streamlined and consistent experience they should deploy whatever Quality of Implementation they see fit to achieve that goal.

§ 4.6. Bit Blasting: Endianness

> what would happen if you did fread into an int? that's my answer 🙂
>
> – Isabella Muerte

It's a simple answer. While we may not be reading into `int`, the idea here is that the interpretation of the directive is meant to encourage and allow for directly copying the bitstream, as faithfully as possible. A compiler-magic based implementation like the ones provided as part of this paper have no endianness issues, but an implementation which writes out integer literals may need to be careful of host vs. target endianness to make sure it serializes correctly to the final binary. As a litmus test, the following code -- given a suitable sized `"foo.bin"` resource -- should return `0`:

```
#include <cstdio>
#include <cstring>

int main() {
        const unsigned char foo0[] = {
#embed "foo.bin"
        };

        const unsigned char foo1[sizeof(foo0)];
        std::FILE* fp = std::fopen("foo.bin");
        if (fp == nullptr) {
                return 1;
        }
        std::size_t foo1_read = std::fread(foo1, 1, sizeof(foo1), fp);
        if (foo1_read != sizeof(foo1)) {
                return 1;
        }
        if (memcmp(&foo0[0], &foo1[0], sizeof(foo0)) != 0) {
                return 1;
        }
        return 0;
}
```

If the same file during both translation and execution, `"foo.bin"`, is used here, this program should always return `0`. This is what the wording below attempts to achieve. Note that this is always a concern already, due to `CHAR_BIT` and other target environment-specific variables that already exist; implementations have always been responsible for handling differences between the host and the

target and this directive is no different. If the `CHAR_BIT` of the host vs. the target is the same, then the directive is more simple. If it is not, then an implementation will have to perform translation.

§ 5. Implementation Experience

An implementation of this functionality is available in branches of both GCC and Clang, accessible right now with an internet connection through the online utility Compiler Explorer. The Clang compiler with this functionality is called "x86-64 clang (std::embed)" in the Compiler Explorer UI.

§ 6. Alternative Syntax

There were previous concerns about the syntax using pragma-like syntax and more. WG14 voted to keep the syntax as a plain `#embed` preprocessor directive, unanimously.

Previously, different syntax was used to specify the limit and other kinds of parameters. These have been normalized to be a suffix of attribute-like parameters.

§ 7. Wording

This wording is relative to C++'s latest working draft.

§ 7.1. Intent

The intent of the wording is to provide a preprocessing directive that:

— takes a quote or chevron delimited header-name -- potentially from the expansion of a macro -- and uses it to find a unique resource in an implementation-defined manner;

— behaves as if it produces a list of values suitable for the initialization of an array as well as initializes each `unsigned char` element according to the specific environment limits found in an implementation-defined manner;

— errors if the size of the resource does not have enough bits to fully and properly initialize all the values generated by the directive;

— allows a limit parameter limiting the number of elements to be specified (but allowing less than the limit);

— produces a core constant expression that can be used to initialize `constexpr` arrays;

— and, present such contents as if it is a list of values, such that it can be used to initialize arrays of known and unknown bound even if additional elements of the whole initialization list come before or after the directive.

§ 7.2. Proposed Feature Test Macro

The proposed feature test macro is `__cpp_pp_embed` for the preprocessor functionality.

§ 7.3. Proposed Language Wording

§ 7.3.1. Append to §14.8.1 Predefined macro names [cpp.predefined] an additional entry:

```
#define __cpp_pp_embed     ?????/* EDITOR VALUE HERE */
```

§ 7.3.2. Add to the *control-line* production in §15.1 Preamble [cpp.pre] a new grammar production, as well as a supporting *embed-attribute-list* production:

embed-parameters:
    attribute$_{opt}$
    embed-parameters attribute$_{opt}$

control-line:
    ...
    `#` `embed` *pp-tokens new-line*

§ 7.3.3. Modify §15.2 Conditional inclusion [cpp.cond] to include a new "has-embed-expression" by modifying paragraph 1 and adding a new paragraph 5 after the current paragraph 4:

has-embed-expression:
    ...
    `__has_embed` **(** *header-name-tokens embed-parameters*$_{opt}$ **)** *new-line*

1 … and it may contain zero or more ~~defined-macro-expressions and/or has-include-expressions and/or has-attribute-expressions as unary operator expressions~~ *defined-macro-expressions*, *has-include-expressions*, *has-attribute-expressions*, and/or *has-embed-expressions* as unary operator expressions .

5 The *resource* identified by the parenthesized preprocessing token sequence in each contained *has-embed-expression* is searched for as if that preprocessing token sequence were the *pp-tokens* in a `# embed` directive ([cpp.res]). If such a directive would not satisfy the syntactic requirements of a `# embed` directive, the program is ill-formed. The *has-embed-expression* evaluates `1` if the search for the *resource* succeeds, or `0` if the search fails.

§ **7.3.4. Add a new sub-clause §15.4 Resource inclusion [cpp.res]:**

15.4 Resource inclusion [**cpp.res**]

1 An `#embed` directive shall identify a *resource* file that can be processed into a comma-delimited list of integer literals where each integer literal is `static_cast` ([expr.static.cast]) to `unsigned char`.

[*Example*:

```
#include <cstddef>

void have_you_any_wool(const unsigned char*, std::size_t);

int main (int, char*[]) {
        constexpr const unsigned char baa_baa[] = {
#embed "black_sheep.ico"
        };

        have_you_any_wool(baa_baa, sizeof(baa_baa));

        return 0;
}
```

– end example]

2 A preprocessing directive of the form

`#` `embed` < *h-char-sequence* > *embed-parameters*$_{opt}$ *new-line*

searches a sequence of implementation-defined places for a *resource* identified uniquely by the specified sequence between the < and > delimiters, and causes the replacement of that directive by a comma-delimited list of integer constant expressions as specified below. How the places are specified or the header identified is implementation-defined. [*Note*: A mechanism similar to, but distinct from, the implementation-defined search paths used for ([cpp.include]) is encouraged. — *end Note*]

3 A preprocessing directive of the form

`#` `embed` " *q-char-sequence* " *embed-parameters*$_{opt}$ *new-line*

searches a sequence of implementation-defined places for a *resource* identified uniquely by the specified sequence between the < and > or the " and " delimiters. How the places are specified or the *resource* identified is implementation-defined. [Note: A mechanism similar to, but distinct from, the implementation-defined search paths used for ([cpp.include]) is encouraged. — end Note] If this search is not supported, or if the search fails, the directive is reprocessed as if it read

`#embed` < *h-char-sequence* > *embed-parameters*$_{opt}$ *new-line*

with the identical contained sequence (including > characters, if any) from the original directive.

4 Either form of the `#embed` directive expands to a comma-separated list of integer literals, with each integer literal `static_cast` to `unsigned char`. Each integer literal shall have an implementation-defined value between `0` and `UCHAR_MAX`, inclusive. If that list is used to initialize a contiguous sequence of `unsigned char`, the elements of the array initialized from the comma-separated list shall contain values as-if `std::fread` ([library.c]) from the *resource* as a file during program execution. [*Note*: Each integer literal produced should closely represent the bit stream of the resource unmodified. This may require an implementation to consider potential differences between translation and execution environments, endianness, and any other applicable sources of mismatch. — *end note*]

[Example:

```
#include <cstring>
#include <cstddef>
#include <fstream>
#include <vector>

int main() {
        const unsigned char d[] = {
#embed <data.dat>
        };
        const std::vector<unsigned char> vec_d = {
#embed <data.dat>
        };

        constexpr std::size_t expected = sizeof(d);
        unsigned char runtime_d[expected];
        std::ifstream f_source("data.dat"); // same file in execution
                                            // as was embedded
        char* ptr = reinterpret_cast<char*>(runtime_d);
```

```
        if (!f_source.read(ptr, expected));
                return 1;

        // if same file as in execution environment, both should be
        // 0 and the byte representations should be identical
        int is_same = std::memcmp(&d[0], ifstream_ptr, ifstream_size)
        int is_same_vec = std::memcmp(vec_d.data(), ifstream_ptr, ifs
        return (is_same == 0 && is_same_vec == 0) ? 0 : 1;
}
```

— end example]

5 An embed directive can take an arbitrary number of *attribute* token sequences after the *q-char-sequence* or *h-char-sequence*, separated by white space. These are its *embed-parameters*, described in ([cpp.res.param]).

6 Let *EMBED-BIT-SIZE* be the implementation-defined size in bits of the contents of the *resource*. The program is ill-formed if *EMBED-BIT-SIZE* is not a multiple of `UCHAR_WIDTH`. A *resource* is considered to be empty if the *EMBED-BIT-SIZE* is zero. If a *resource* is empty, then the `# embed` directive expands to nothing.

[*Example*:

```
int main (int, char*[]) {
        const unsigned char coeffs[] = {
// ill-formed: EMBED-BIT-SIZE of 6 is too small for unsigned char
#embed "6_bits.bin"
        };

        const unsigned char fac[] = {
// may be ill-formed: EMBED_BIT_SIZE % UCHAR_WIDTH may not be 0
// on a system where the resource has an implementation-defined
// bit size of 12 bits
#embed "12_bits.bin"
        };

        return 0;
}
```

– end example]

7 A preprocessing directive of the form

 # embed *pp-tokens new-line*

(that does not match one of the two previous forms) is permitted. The preprocessing tokens after **embed** in the directive are processed just as in normal text. (Each identifier currently defined as a macro name is replaced by its replacement list of preprocessing tokens.) The directive resulting after all replacements shall match one of the two previous forms [Note: Adjacent string literals are not concatenated into a single string literal; thus, an expansion that results in two string literals is an invalid directive. — end Note]. The method by which a sequence of preprocessing tokens between a < and a > preprocessing token pair or a pair of " characters is combined into a single *resource* name preprocessing token is implementation-defined.

[*Example*:

```
#define INT_DATA_H "i.dat"

int main () {
        int i = {
#embed INT_DATA_H
        }; // well-formed if i.dat produces 1 value, i value is [0, U
        struct s {
                double a, b, c;
                struct { double e, f, g; };
                double h, i, j;
        };
        s x = {
        // well-formed, initializes each element in
        // order according to initialization rules for a
        // brace-delimited, comma-separated list
#embed "s.dat"
        };
        return 0;
}
```

– end example]

15.4.1 Parameters [cpp.res.param]

1 The *embed-parameters* contain *attributes* which may modify the result of the replacement for the `# embed` preprocessing directive. The *attribute-tokens* defined below are `limit`, `prefix`, and `suffix`.

2 For an *attribute-token* (including an *attribute-scoped-token*) not specified in this clause, the behavior is implementation-defined. Tokens reserved as *embed-parameters* for future revisions of this document and implementations are as described in ([dcl.attr.grammar]).

3 Prior to evaluation, macro invocations in the list of preprocessing tokens that may become the *embed-parameters* are replaced, just as in normal text. If tokens are generated that match an *embed-parameter*'s *attribute-token* or *attribute-scoped-token* (defined either below or by the implementation) as a result of this replacement process or its use does not match one of the two specified forms prior to macro replacement, the behavior is undefined.

15.4.1.1 Limit parameter [**cpp.res.param.limit**]

1 The *attribute-token* `limit` denotes a maximum number of elements that may be produced in the comma delimited list. It may appear zero, one, or multiple times in the *embed-parameters* list. The most recent *attribute* in lexical order applies and the others are ignored. It's *attribute-argument-clause* shall be present and have the form:

(*pp-tokens* **)**

2 It's *attribute-argument-clause* shall be a positive integral constant expression after evaluation, and be processed as described in conditional inclusion ([cpp.cond]).

3 Let *N* be the evaluation of the *attribute-argument-clause* as specified above. If *EMBED-BIT-SIZE* is not a multiple of `UCHAR_WIDTH` and *EMBED-BIT-SIZE* is less than `UCHAR_WIDTH` multiplied by `N`, then the program is ill-formed. [*Note*: This requirement supersedes the previous constraint that *EMBED-BIT-SIZE* shall be an exact multiple of `UCHAR_WIDTH`. — *end Note*]

[*Example*:

```
#include <cassert>

int main (int, char*[]) {
        constexpr const char sound_signature[] = {
#embed <sdk/jump.wav> limit(2+2)
        };

        // verify PCM WAV resource
        assert(sound_signature[0] == 'R');
        assert(sound_signature[1] == 'I');
        assert(sound_signature[2] == 'F');
        assert(sound_signature[3] == 'F');
        assert(sizeof(sound_signature) == 4);
```

```
        return 0;
}
```

– end example]

[*Example*:

```
int main (int, char*[]) {
        const unsigned char scal[] = {
// may be ill-formed: if UCHAR_WIDTH is greater than 24,
// this may issue a diagnostic if (UCHAR_WIDTH * 1) % 24
// is not equivalent to 0
#embed "24_bits.bin" limit(1)
        };

        return 0;
}
```

– end example]

15.4.1.2 Prefix parameter [cpp.res.param.prefix]

1 The *attribute-token* `prefix` denotes a maximum number of elements that may be produced in the comma delimited list. It may appear zero, one, or multiple times in the *embed-parameters* list. The most recent *attribute* in lexical order applies and the others are ignored. It's *attribute-argument-clause* shall be present and have the form:

(*pp-tokens*$_{opt}$ **)**

2 If the *resource* is empty, this *embed-parameter* is ignored. Otherwise, any *pp-tokens* specified shall be placed immediately before the expansion of the `# embed` directive.

15.4.1.3 Suffix parameter [cpp.res.param.suffix]

1 The *attribute-token* `suffix` denotes a maximum number of elements that may be produced in the comma delimited list. It may appear zero, one, or multiple times in the *embed-parameters* list. The most recent *attribute* in lexical order applies and the others are ignored. It's *attribute-argument-clause* shall be present and have the form:

(*pp-tokens*$_{opt}$ **)**

2 If the *resource* is empty, this *embed-parameter* is ignored. Otherwise, any *pp-tokens* specified shall be placed directly after the expansion of the `# embed` directive.

[*Example*:

```
#include <cstring>
#include <cassert>

#ifndef SHADER_TARGET
#define SHADER_TARGET "ches.glsl"
#endif

extern char* merp;

void init_data () {
        constexpr const char whl[] = {
#embed SHADER_TARGET \
                prefix(0xEF, 0xBB, 0xBF, ) /* UTF-8 BOM */ \
                suffix(,)
                0
        };
        // always null terminated,
        // contains BOM if not-empty
        bool is_good = (sizeof(whl) == 1 && whl[0] == '\0')
        || (whl[0] == '0xEF' && whl[1] == '0xBB'
            && whl[2] == '0xBF' && whl[sizeof(whl) - 1] == '\0');
        assert(is_good);
        std::strcpy(merp, whl);
}
```

– end example]

15.4.1.4 Empty parameter [**cpp.res.param.empty**]

1 The *attribute-token* `empty` denotes a sequence of tokens to use if the *resource* is empty. It may appear zero, one, or multiple times in the *embed-parameters* list. The most recent *attribute* in lexical order applies and the others are ignored. It's *attribute-argument-clause* shall be present and have the form:

(*pp-tokens*$_{opt}$)

2 If the *resource* is not empty, this *embed-parameter* is ignored. Otherwise, any *pp-tokens* specified shall be placed where the embed directive is.

[*Example*: If the file is empty, then this

```
constexpr const char x[] = {
#embed "empty_file.dat" \
        empty((char)-1)
};
```

expands to

```
constexpr const char x[] = {
        (char)-1
};
```

. Otherwise, it expands to the contents of the file. – *end example*]

§ 8. Acknowledgements

Thank you to Alex Gilding for bolstering this proposal with additional ideas and motivation. Thank you to Aaron Ballman, David Keaton, and Rajan Bhakta for early feedback on this proposal. Thank you to the `#include<C++>` for bouncing lots of ideas off the idea in their Discord. Thank you to Hubert Tong for refining the proposal's implementation-defined extension points.

Thank you to the Lounge<C++> for their continued support, and to rmf for the valuable early implementation feedback.

§ 9. Appendix

§ 9.1. Existing Tools

This section categorizes some of the platform-specific techniques used to work with C++ and some of the challenges they face. Other techniques used include pre-processing data, link-time based tooling, and assembly-time runtime loading. They are detailed below, for a complete picture of today's landscape of options. They include both C and C++ options.

§ 9.1.1. Pre-Processing Tools

1. Run the tool over the data (`xxd -i xxd_data.bin > xxd_data.h`) to obtain the generated file (`xxd_data.h`) and add a null terminator if necessary:

```
unsigned char xxd_data_bin[] = {
        0x48, 0x65, 0x6c, 0x6c, 0x6f, 0x2c, 0x20, 0x57, 0x6f, 0x72, 0x6c, 0
        0x0a, 0x00
};
unsigned int xxd_data_bin_len = 13;
```

2. Compile `main.c`:

```
#include <stdlib.h>
#include <stdio.h>

// prefix as const,
// even if it generates some warnings in g++/clang++
const
#include "xxd_data.h"

int main() {
                const char* data = reinterpret_cast<const char*>(xxd_data_b
                puts(data); // Hello, World!
                return 0;
}
```

Others still use python or other small scripting languages as part of their build process, outputting data in the exact C++ format that they require.

There are problems with the `xxd -i` or similar tool-based approach. Tokenization and Parsing data-as-source-code adds an enormous overhead to actually reading and making that data available.

Binary data as C(++) arrays provide the overhead of having to comma-delimit every single byte present, it also requires that the compiler verify every entry in that array is a valid literal or entry according to the C++ language.

This scales poorly with larger files, and build times suffer for any non-trivial binary file, especially when it scales into Megabytes in size (e.g., firmware and similar).

9.1.2. `python`

Other companies are forced to create their own ad-hoc tools to embed data and files into their C++ code. MongoDB uses a custom python script, just to format their data for compiler consumption:

```
import os
import sys

def jsToHeader(target, source):
                outFile = target
                h = [
                                '#include "mongo/base/string_data.h"',
                                '#include "mongo/scripting/engine.h"',
                                'namespace mongo {',
                                'namespace JSFiles{',
                ]
                def lineToChars(s):
                                return ','.join(str(ord(c)) for c in (s.rst
                for s in source:
                                filename = str(s)
                                objname = os.path.split(filename)[1].split(
                                stringname = '_jscode_raw_' + objname

                                h.append('constexpr char ' + stringname + '

                                with open(filename, 'r') as f:
                                                for line in f:
                                                                h.append(li

                                h.append("0};")
                                # symbols aren't exported w/o this
                                h.append('extern const JSFile %s;' % objnam
                                h.append('const JSFile %s = { "%s", StringD
                                                                (objname,

                h.append("} // namespace JSFiles")
                h.append("} // namespace mongo")
                h.append("")

                text = '\n'.join(h)

                with open(outFile, 'wb') as out:
                                try:
```

```
                                        out.write(text)
                                finally:
                                        out.close()


if __name__ == "__main__":
                if len(sys.argv) < 3:
                                print "Must specify [target] [source] "
                                sys.exit(1)
                jsToHeader(sys.argv[1], sys.argv[2:])
```

MongoDB were brave enough to share their code with me and make public the things they have to do: other companies have shared many similar concerns, but do not have the same bravery. We thank MongoDB for sharing.

9.1.3. `ld`

A complete example (does not compile on Visual C++):

1. Have a file ld_data.bin with the contents `Hello, World!`.
2. Run `ld -r binary -o ld_data.o ld_data.bin`.
3. Compile the following `main.cpp` with `gcc -std=c++17 ld_data.o main.cpp`:

```
#include <stdlib.h>
#include <stdio.h>

#define STRINGIZE_(x) #x
#define STRINGIZE(x) STRINGIZE_(x)

#ifdef __APPLE__
#include <mach-o/getsect.h>

#define DECLARE_LD_(LNAME) extern const unsigned char _section$__DATA__##LN
#define LD_NAME_(LNAME) _section$__DATA__##LNAME
#define LD_SIZE_(LNAME) (getsectbyLNAME("__DATA", "__" STRINGIZE(LNAME))->s
#define DECLARE_LD(LNAME) DECLARE_LD_(LNAME)
#define LD_NAME(LNAME) LD_NAME_(LNAME)
#define LD_SIZE(LNAME) LD_SIZE_(LNAME)
```

```
#elif (defined __MINGW32__) /* mingw */

#define DECLARE_LD(LNAME)                                   \
        extern const unsigned char binary_##LNAME##_start[];    \
        extern const unsigned char binary_##LNAME##_end[];
#define LD_NAME(LNAME) binary_##LNAME##_start
#define LD_SIZE(LNAME) ((binary_##LNAME##_end) - (binary_##LNAME##_start))
#define DECLARE_LD(LNAME) DECLARE_LD_(LNAME)
#define LD_NAME(LNAME) LD_NAME_(LNAME)
#define LD_SIZE(LNAME) LD_SIZE_(LNAME)

#else /* gnu/linux ld */

#define DECLARE_LD_(LNAME)                                    \
        extern const unsigned char _binary_##LNAME##_start[];     \
        extern const unsigned char _binary_##LNAME##_end[];
#define LD_NAME_(LNAME) _binary_##LNAME##_start
#define LD_SIZE_(LNAME) ((_binary_##LNAME##_end) - (_binary_##LNAME##_start
#define DECLARE_LD(LNAME) DECLARE_LD_(LNAME)
#define LD_NAME(LNAME) LD_NAME_(LNAME)
#define LD_SIZE(LNAME) LD_SIZE_(LNAME)
#endif

DECLARE_LD(ld_data_bin);

int main() {
        const char* p_data = reinterpret_cast<const char*>(LD_NAME(ld_data_
        // impossible, not null-terminated
        //puts(p_data);
        // must copy instead
        return 0;
}
```

This scales a little bit better in terms of raw compilation time but is shockingly OS, vendor and platform specific in ways that novice developers would not be able to handle fully. The macros are required to erase differences, lest subtle differences in name will destroy one's ability to use these macros effectively. We omitted the code for handling VC++ resource files because it is excessively verbose than what is present here.

N.B.: Because these declarations are `extern`, the values in the array cannot be accessed at compilation/translation-time.

9.1.4. `incbin`

There is a tool called `incbin` which is a 3rd party attempt at pulling files in at "assembly time". Its approach is incredibly similar to `ld`, with the caveat that files must be shipped with their binary. It unfortunately falls prey to the same problems of cross-platform woes when dealing with Visual C, requiring additional pre-processing to work out in full.

9.2. Type Flexibility

Note: As per the vote in the September C++ Evolution Working Group Meeting, Type Flexibility is not being pursued in the preprocessor for various implementation and support splitting concerns.

A type can be specified after the `#embed` to view the data in a very specific manner. This allows data to initialized as exactly that type.

Type flexibility was not pursued for various implementation concerns. Chief among them was single-purpose preprocessors that did not have access to frontend information. This meant it was very hard to make a system that was both preprocessor conformant but did not require e.g. `sizeof(...)` information at the point of preprocessor invocation. Therefore, the type flexibility feature was pulled from `#embed` and will be conglomerated in other additions such as `std::bitcast` or `std::embed`.

```
/* specify a type-name to change array type */
const int shorten_flac[] = {
                #embed int "stripped_music.flac"
};
```

The contents of the resource are mapped in an implementation-defined manner to the data, such that it will use `sizeof(type-name) * CHAR_BIT` bits for each element. If the file does not have enough bits to fill out a multiple of `sizeof(type-name) * CHAR_BIT` bits, then a diagnostic is required. Furthermore, we require that the type passed to `#embed` that must one of the following fundamental types, signed or unsigned, spelled exactly in this manner:

— `char`, `unsigned char`, `signed char`

— `short`, `unsigned short`, `signed short`

— `int`, `unsigned int`, `signed int`

— `long`, `unsigned long`, `signed long`

— `long long`, `unsigned long long`, `signed long long`

More types can be supported by the implementation if the implementation so chooses (both the GCC and Clang prototypes described below support more than this). The reason exactly these types are required is because these are the only types for which there is a suitable way to obtain their size at pre-processor time. Quoting from §5.2.4.2.1, paragraph 1:

> The values given below shall be replaced by constant expressions suitable for use in `#if` preprocessing directives.

This means that the types above have a specific size that can be properly initialized by a preprocessor entirely independent of a proper C frontend, without needing to know more than how to be a preprocessor. Originally, the proposal required that every use of `#embed` is accompanied by a `#include <limits.h>` (or, in the case of C++, `#include <climits>`). Instead, the proposal now lets the implementation "figure it out" on an implementation-by-implementation basis.